



DAY 1 · SESSION;

Date & Time: 11-Aug-2026 @ 10:30 AM

Presenters: Chandra Sekhar & Ravali

Python for Data Engineering

The parts you will use every day for the next five weeks

Data structures

Comprehensions

Functions & scope

Type hints

OOP & dunder

Exceptions

Context managers

pytest

Lab -Demo

What you will be able to do

1

Pick the right data structure

and say why, in complexity terms, not by habit

2

Write comprehensions that help

and recognise the ones that hurt readability

3

Avoid the default-argument trap

the bug that silently shares state between calls

4

Type-hint every public function

so the next person does not have to read the body

5

Use dataclasses and dunders

instead of writing boilerplate by hand

6

Catch exceptions by type

and explain why a bare except is a bug, not a style choice

7

Write a context manager

and know what it guarantees when things go wrong

8

Write tests that would fail

if the code were wrong — which is the only kind worth writing

Data structures — pick by access pattern, not by habit

Type	Properties	Cost that matters	Reach for it when
list	Ordered, mutable, duplicates allowed	$O(1)$ append · $O(n)$ lookup by value	A sequence you iterate or append to
tuple	Ordered, immutable, hashable if contents are	$O(1)$ index	A fixed record, or a dict key
set	Unordered, unique, hashable members	$O(1)$ membership	Deduplication and 'is this in that'
dict	Key to value, insertion-ordered since 3.7	$O(1)$ lookup by key	Lookup by a business key
frozenset	Immutable set	$O(1)$ membership	A constant set of allowed values

THE ONE THAT BITES

```
# 50,000 order ids, checked in a loop
seen = []                # list
if oid in seen:          #  $O(n)$  each time
    ...                  # ~30 seconds

seen = set()             # set
if oid in seen:          #  $O(1)$  each time
    ...                  # ~0.02 seconds
```

The interview question behind this slide

- “Why is this loop slow?” — the answer is almost always a list where a set belongs
- “Can a tuple be a dict key?” — yes, if everything inside it is hashable. Put a list inside and it cannot
- “Is a dict ordered?” — insertion order is guaranteed from 3.7. Do not rely on it for sorting

Comprehensions — and when not to use one

THE FOUR FORMS

```
# list, set, dict, generator
names = [r['name'] for r in rows]
unique = {r['region'] for r in rows}
lookup = {r['id']: r for r in rows}
stream = (clean(r) for r in rows) # lazy

# filtering and transforming together
totals = [
    r['qty'] * r['price']
    for r in rows
    if r['qty'] > 0
]
```

WHEN TO STOP

```
# Don't. Nobody can read this.
out = [f(x) if p(x) else g(x)
       for sub in data
       for x in sub
       if q(x) and r(x)]

# Do. A loop is not a failure.
out = []
for sub in data:
    for x in sub:
        if q(x) and r(x):
            out.append(f(x) if p(x) else g(x))
```

The rule: one loop and one condition is a comprehension. Two loops, or a condition plus a conditional expression, is a for loop. Readability is a performance characteristic of the team, not of the code.

THE ONE THAT MATTERS FOR US

```
# Generators matter for data work: this never holds the whole file in memory
total = sum(r.line_total for r in clean_rows(read_raw_csv(path))) # streams row by row
```

Functions and scope — and the trap that will bite you

THE DEFAULT-ARGUMENT TRAP

```
def add_row(row, batch=[]):    # BUG
    batch.append(row)
    return batch
```

```
add_row('a')    -> ['a']
add_row('b')    -> ['a', 'b']    # !!
```

```
# The default is evaluated ONCE,
# when the function is defined.
# Every call shares that one list.
```

THE FIX, EVERY TIME

```
def add_row(row, batch=None):  # FIX
    if batch is None:
        batch = []
    batch.append(row)
    return batch
```

```
# Rule: never use a mutable default.
# list, dict, set — all unsafe.
# Immutables are fine: 0, '', None,
# and tuples.
```

LEGB — how a name is resolved

- Local — inside this function
- Enclosing — the function that wraps it
- Global — module level
- Built-in — len, sum, print
- Assigning to a name makes it local for the whole function, even before the assignment line

MAKE CALL SITES READABLE

```
# Keyword-only arguments: use them
def normalise_text(
    raw: str | None,
    *,                          # everything after
    title_case: bool = False,
) -> str | None: ...

normalise_text(x, True)          # rejected
normalise_text(x, title_case=True) # clear
```

Modules, packaging and environments with uv

THE FIVE COMMANDS YOU NEED

```
$ uv init sales-cleaner
$ uv add --dev pytest ruff mpy
$ uv sync --extra dev
$ uv run pytest
$ uv run clean-sales in.csv out.csv

# uv replaces pip + venv + pip-tools.
# One lockfile, reproducible, fast.
# No 'works on my machine'.
```

THE SRC LAYOUT WE USE

```
src/sales_cleaner/
    __init__.py    # public surface
    parsing.py     # field parsers
    cleaner.py     # row logic + IO
    cli.py         # args and printing only
tests/
    conftest.py    # shared fixtures
    test_parsing.py
    test_cleaner.py
```

Why src/ and why a thin CLI

- src/ layout means the tests import the installed package, not the folder next to them — so a packaging mistake fails in your test run rather than in production
- The CLI does argument parsing and printing. Nothing else. Everything it does is available as a function, which is what makes the logic testable without shelling out
- __init__.py declares the public surface. If it is not exported there, callers should not be importing it

Type hints — write them for the reader, not the compiler

WHAT THEY BUY YOU

```
# Without hints: read the body to know
def parse_amount(raw):
    ...

# With hints: the signature is the doc
def parse_amount(
    raw: str | None,
) -> Decimal | None:
    ...

# It can take None. It can return None.
# Both facts now impossible to miss.
```

THE SYNTAX YOU WILL USE

```
from __future__ import annotations

names: list[str]
lookup: dict[str, CleanRow]
formats: tuple[str, ...]
maybe: str | None          # 3.10+
rows: Iterable[dict[str, str]]

# Union return forces the caller
# to handle both outcomes:
def clean_row(...) -> CleanRow | RejectedRow:
```

Three rules

- Hint every public function. Internal helpers are optional but you will want them anyway
- Python does not enforce hints at runtime — mypy does, in CI. A hint nobody checks is a comment that goes stale
- If a hint is hard to write, the function is probably doing two things. That is the hint telling you something

OOP and dunder methods — let the language write the boilerplate

DATACLASS: 6 LINES INSTEAD OF 40

```
@dataclass(frozen=True, slots=True)
class CleanRow:
    order_id: str
    order_date: date
    quantity: int
    unit_price: Decimal

    @property
    def line_total(self) -> Decimal:
        return self.unit_price * self.quantity

# You get __init__, __repr__, __eq__ free.
# frozen -> immutable. slots -> less memory.
```

The dunders worth knowing

<code>__init__</code>	construction
<code>__repr__</code>	what you see when debugging
<code>__eq__</code>	what <code>==</code> means
<code>__hash__</code>	usable as a dict key or in a set
<code>__enter__</code> / <code>__exit__</code>	context manager
<code>__iter__</code> / <code>__next__</code>	iteration
<code>__len__</code>	<code>len()</code> works
<code>__str__</code>	what <code>print()</code> shows

When a class, and when not

- A class that has no state and one method should have been a function. This is the most common over-engineering in data code
- A frozen dataclass is the right default for a record. If something downstream needs to mutate it, that is worth an argument in review
- Inheritance is rarely the answer in pipeline code. Composition and plain functions almost always read better

The exception hierarchy — and why bare except is a bug

WHAT SITS WHERE

BaseException

```

├── SystemExit      # sys.exit()
├── KeyboardInterrupt # Ctrl-C
├── GeneratorExit
├── Exception       # catch here
│   ├── ValueError
│   │   └── UnicodeDecodeError
│   ├── TypeError
│   ├── KeyError / IndexError
│   ├── OSError -> FileNotFoundError
│   └── ArithmeticError

```

THE ONE RULE

```

try:                                # BUG
    value = parse(raw)
except:                             # catches Ctrl-C,
    value = None                    # SystemExit,
                                    # and your typos

try:                                # FIX
    value = Decimal(text)
except InvalidOperation:            # by type
    return None                     # and only this

```

Why this matters more in data work than anywhere else

- A bare except around a parse turns a NameError in your own code into a silently skipped row. The pipeline stays green and the numbers are wrong
- Catch the narrowest exception that can actually happen. If you cannot name it, you do not yet know what the code does

Context managers — guarantee the cleanup

THE FORM YOU USE DAILY

```
# The file closes even if this raises
with path.open(newline='',
                encoding='utf-8-sig') as fh:
    reader = csv.DictReader(fh)
    for row in reader:
        yield normalise(row)

# utf-8-sig strips the BOM. You will need
# this exact line in Lab 1.
```

WRITE YOUR OWN

```
from contextlib import contextmanager

@contextmanager
def timed(label: str):
    start = time.perf_counter()
    try:
        yield
    finally:
        print(f'{label}: {...:.2f}s')

with timed('clean'):
    result = clean_file(path)
```

What a context manager actually guarantees

- `__exit__` runs whether the block finished, returned early, or raised. That is the whole point — the guarantee is unconditional
- The finally in `@contextmanager` is doing the same job. Leave it out and an exception skips your cleanup
- This is why we never write `open()` without `with`. Not style — a file handle leaked in a long-running job eventually exhausts the descriptor limit
- Databricks and Spark sessions, database connections and temp directories all follow the same pattern. Learn it once here

pytest — the fundamentals

PLAIN ASSERT, NO FRAMEWORK NOISE

```
# tests/test_parsing.py
def test_parse_amount_strips_separators():
    assert parse_amount('1,200.50') == \
        Decimal('1200.50')

def test_returns_none_not_raises():
    assert parse_amount('rubbish') is None

$ uv run pytest           # all
$ uv run pytest -k amount # by name
$ uv run pytest -x        # stop at 1st
```

PARAMETRIZE: ONE TEST, MANY CASES

```
@pytest.mark.parametrize(
    ('raw', 'expected'),
    [
        ('1,200.50', Decimal('1200.50')),
        ('(1200.00)', Decimal('-1200.00')),
        ('1200.00-', Decimal('-1200.00')),
        ('0', Decimal('0')),
    ],
)
def test_amounts(raw, expected):
    assert parse_amount(raw) == expected
# -> four separate named test results
```

Fixtures

- A fixture is setup shared between tests, declared once in `conftest.py`
- `tmp_path` is built in: each test gets its own throwaway directory
- Request one by naming it as an argument. No wiring, no `setUp` method

What makes a test worth writing

- It would fail if the code were wrong. A test that passes either way is worse than none
- It names the behaviour, not the function. `test_zero_is_a_value_not_a_null`
- It tests the edge, not the happy path. The happy path rarely breaks

Lab 1 — clean the messy sales file

Read a deliberately messy CSV, clean it, write the output, with five pytest tests covering the edge cases. Submitted as a pull request and reviewed by a peer before it counts as done.

BOM + messy headers	order_id, 'Order Date', ' customer_name '
Six date formats	2024-01-15 · 15/01/2024 · Jan 17 2024 · 17 Jan 2024
Money as text	"1,200.50" · (1200.00) · 1200.00-
Six spellings of nothing	empty · NA · N/A · NULL · - · ?
Casing and spacing	' ravi kumar ' · 'PRIYA SHARMA' · 'SOUTH'
Valid but wrong	quantity = -2 · missing order_id
A correction	ORD-1001 appears twice — the later one wins

Done means merged, not written

Tests green · ruff clean · CLI runs end to end · every rejected row has a reason a non-engineer could act on · no bare except · PR reviewed by a peer and merged.

```
HOW TO START

uv sync --extra dev
uv run pytest                # all failing
# read a failing test, write the function
uv run pytest -k amount
uv run ruff check .
uv run clean-sales \
    data/messy_sales.csv out/clean.csv

in=14 clean=4 rejected=9 duplicates=1

# The rule: in == clean + rejected + dupes
# Nothing is dropped silently. Ever.
```

BEFORE YOU START

Five things that will come up in review

1

Money is a Decimal, never a float

$0.1 + 0.2 \neq 0.3$ and finance will notice

2

No bare except, anywhere

Catch by type or do not catch

3

None means absent, not zero

And 0 is a value, not a null

4

Reject rows, do not drop them

Every reject carries a reason and a line number

5

Read the failing test first

The test is the spec. Write the function to it, not around it

Q&A

Thank You